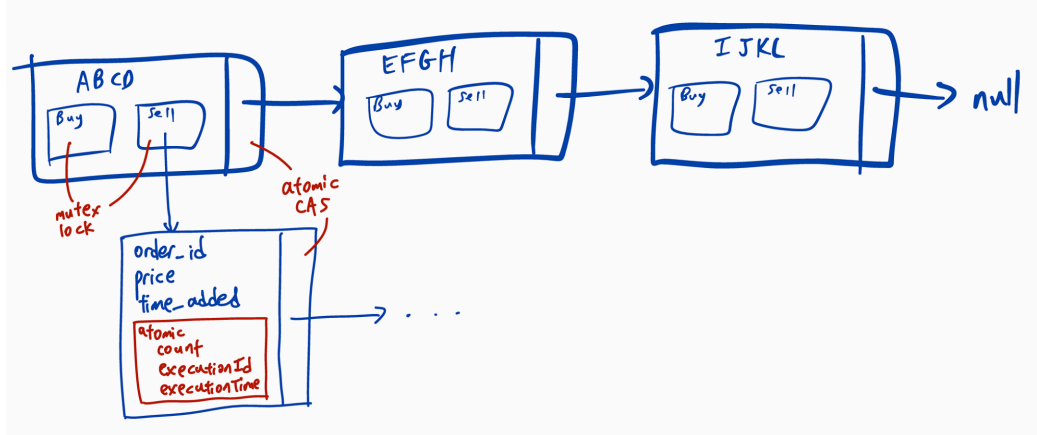


Overview

The matching engine developed can handle up to 40 clients which interact with the engine through buy, sell and cancel orders. The active order interacts with orders in the order book, if the order cannot be completed, it is then added to the order book.

Description of Data Structures

A linked list based structure was used, with the `InstrumentOrderBook` as the main list. Each node represents an instrument and contains two sub linked lists for buy and sell orders. An `unordered_map` was initialised per client to track their order IDs, associated instruments, and order types, improving traversal for cancel orders, particularly with the buy/sell turn lock.



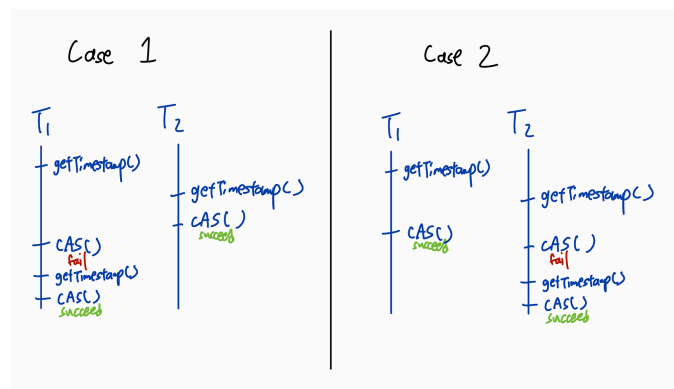
Synchronisation Primitives Used

Atomic Linked List for Thread Safety

The linked list uses atomic `next` and `head` pointers with Compare-and-Swap (CAS) operations for lock-free updates, enabling concurrent access by multiple threads. Memory order was chosen based on the operation: relaxed memory order was used for intermediate steps, while acquire release ensured visibility of final updates (e.g. order counts) to other threads. While traversing to insert two nodes concurrently, the CAS update ensures that only one node is added to the tail, while the `traverse` function allows flexibility to continue traversal after the node was added in order to add an additional node (e.g. see `AtomicList::getOrInsert`).

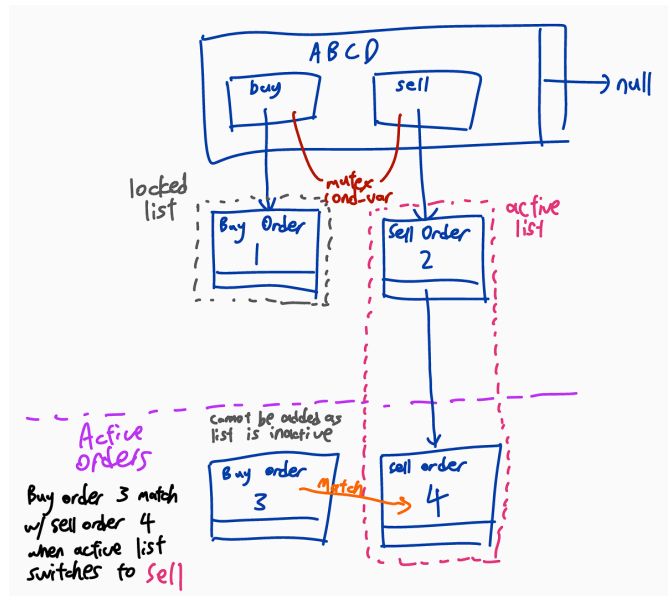
Thread-Safe Order Data

Only `count` and `executionId` are updated by multiple threads, while other fields like `orderId` are read only and require no synchronization. These variables were grouped into a single struct and updated atomically using CAS, ensuring consistency. The `executionTime` was included in this struct to provide accurate timestamps for clients by guaranteeing a happens-before.



Synchronizing Active Buys and Sells with Mutex and Condition Variable

To prevent invalid scenarios - specifically, active buy and sell orders being added to the order book without matching when they are supposed to match each other - a mutex and condition variable were used to ensure mutual exclusion between processing active buys and sells. It is acknowledged that while this prevents data races, it could also lead to starvation (e.g. active sells being delayed during active buy processing). A potential improvement would be to implement an arbitrary time limit of 100ms to balance fairness and concurrency, though this would reduce overall throughput.



Proposed Level of Concurrency: Phase-Level

Order concurrency for different Instruments

Since each instrument is represented by a distinct linked list node in the `InstrumentOrderBook`, orders for a specific instrument only perform read and write operations within that instrument's node. These operations do not interfere with the buy/sell linked lists of other instruments. However, concurrency is temporarily restricted when inserting new instruments into the `InstrumentOrderBook`. If active orders reference more than one instrument which are not yet present, CAS operations are used to insert them into the linked list, where only one insertion can succeed at a time. Subsequent order operations for different instruments can only proceed concurrently once their respective instrument nodes are created.

Order concurrency within the same instrument

Within the same instrument, multiple buy or sell orders from different clients can be processed concurrently. However, buy and sell orders cannot occur simultaneously. Cancel orders, on the other hand, can be executed even when a buy or sell operation is in progress.

1. Concurrent processing of multiple same type orders

Multiple orders of the same type can be processed concurrently using CAS operations. For instance, where there are multiple active buy orders, each active order searches the resting sell list to find the best match based on price and timestamp. If multiple active buy orders attempt to execute on the same resting sell order, the CAS operation that succeeds first determines the execution outcome. If there is remaining quantity after the CAS operation, subsequent active buy orders can continue attempting to execute. Once the quantity of the resting sell order reaches zero, the buy order will move on to find the next best match. This process repeats until

all active buy orders cannot find a valid match. Incomplete or unmatched buy orders are then placed back into the resting list.

2. Handling active orders of different types

When a new active order arrives for a particular instrument, it must ensure that no opposing active orders are currently being processed. This is done using the Lock + Condition variable explained above. For example, if a buy order is received, it must wait until all active sell orders have completed before proceeding. Once all sell orders have been processed, a condition variable notifies all waiting threads, allowing buy orders to resume execution.

3. Fully concurrent cancel orders

Cancel orders can be processed concurrently with active buy or sell orders. Since clients can only cancel their own orders, a sequenced-before relationship is guaranteed. This ensures that an order is first placed into the resting list before a cancel request can act on it. If the targeted order has already been fulfilled as an active order, it will no longer exist in the resting list, making the cancel request invalid.

If a cancel order targets a resting order that is also being matched against active orders, both the cancel and active orders will perform CAS operations on it. If the cancel order's CAS operation succeeds (by atomically setting its count to 0), the resting order is successfully canceled, and the active order will move on to find the next best match. However, if the active order's CAS operation succeeds first, the cancel order will still attempt to cancel the remaining quantity if it has not yet been fully matched. If the resting order is entirely fulfilled before the cancel operation takes effect, the cancel request becomes invalid.

Limitations

Apart from the previously mentioned buy/sell lock starvation, the engine may time out on long test cases due to undeleted nodes when an order's quantity reaches zero. This is because of unnecessary traversals of redundant nodes during match searches or cancellations. Implementing a node recycling linked list could improve efficiency by reusing nodes and reducing allocation overhead, but doing so would require additional locks which may slow down other processes.

Explanation of testing methodology

The input format from the grader's README and the assignment description were fed into ChatGPT to draft a Python script (See [scripts/gen_tests.py](#)) that generated randomised inputs for up to 40 clients and ensured adherence to the assignment's logic. Randomisation was achieved by applying varying weights to the wait, cancel, and order commands. Repeated testing with 100,000 actions and 40 clients helped identify concurrency issues. A bash script (See [scripts/test.sh](#)) was also created to automate test execution and display results. When errors occurred, hypotheses were formed, and smaller test cases were crafted to reproduce the issues. For instance, one of the small test cases helped identify duplicate instruments in the `InstrumentOrderBook` (See [scripts/test1.in](#)).